2018 28th International Telecommunication Networks and Applications Conference (ITNAC)

Cloud Applications Consolidation through Context Information and Heuristic Optimization

Alessandro Carrega, Matteo Repetto
CNIT — S3ITI Lab, Italy
{*alessandro.carrega*, *matteo.repetto*}@cnit.it

***Abstract*—Resource consolidation has been proposed as an effective mechanism to save energy in data centers. Several algorithms have been developed for this purpose, which use the minimal number of servers and network switches, and power off unused equipment. Consolidation algorithms usually take into account static service constraints (Central Processing Unit – CPU, Random Access Memory – RAM, disk, bandwidth), but do not consider dynamic context information, as CPU utilization and the specific role of each Virtual Machines (VMs) in the running application (e.g., core component, backup replica, member of a pool of workers for load balancing).**

In this paper, we describe and evaluate a novel heuristic-based consolidation strategy that explicitly considers context information. Our approach avoids running idle VMs that are pre-provisioned for availability and redundancy purposes, hence pursuing a better linear relationship between power consumption and actual computation than other existing algorithms. We demonstrate through simulations, by comparing different heuristics, the optimal trade-off between service level and energy efficiency achieved by our approach.

I. Introduction

Efficient usage of data centers requires modulating their power consumption according to the actual workload, targeting an ideal linear behaviour [1]. Enabling power-saving mechanisms to this aim include frequency/voltage scaling, and sleep states [2]. *Sleeping*[1] is by far the most efficient mechanism to save energy [2], but it could only be applied to idle devices. Hence, resource consolidation has been proposed to maximize the effectiveness of such procedure, by clustering applications together on the smallest number of servers according to performance constraints, so that all the remaining idle infrastructure could be put to sleep.

Simplest strategies for consolidation cluster applications without taking into account power consumption of the physical devices (i.e., they are *power-unaware*), whereas more advanced algorithms also consider the energy drawn by servers, network equipment as well as Heating, Ventilation and Air Conditioning (HVAC) systems [2]. Unfortunately, consolidation entails the need to migrate applications among servers, hence discontinuing operation for a short time, which may be unacceptable for mission-critical applications [3].

Consolidation algorithms usually operate under service-level constraints on Central Processing Unit (CPU), Random Access Memory (RAM), disk, bandwidth. This usually results in optimal placement plans that guarantee the required performance level, sometimes only on statistical bases (i.e., with some degree of over-commitment). Though this approach greatly enhances the efficiency of the infrastructure, it cannot achieve a straight linear relationship between utilization and power consumption, because it does not consider the real usage of Virtual Machines (VMs) [4, 5]. As a matter of fact, large cloud applications can be composed of many VMs, some of which just provisioned for scalability and redundancy purposes (i.e., to effectively cope with workload fluctuations and service/infrastructure failures), which run idle most of the time. This is especially necessary for real-time critical services, since the provisioning process may take several minutes or hours and even days for administrative matters. It is clear that this situation leads to inefficiency for infrastructure owners and higher operational costs for service providers.

In this paper, we describe and evaluate a consolidation strategy based on an enhanced system model for the Infrastructure as a Service (IaaS) cloud paradigm, which takes into consideration both servers and network equipment. The main novelty of our approach is the explicit consideration of context information, by *labeling* VMs according to their sensitivity to power-saving mechanisms (i.e., over-commitment, live migration, suspend-to-ram). In practice, this means the ability to take into account their current role in the application they are part of. With such an approach, our algorithm enables infrastructure providers to selectively use aggressive power saving mechanisms (i.e., sleep states) for servers that only run idle VMs, bringing them back to full operation in a few hundred milliseconds [6]. This strategy allows achieving a better trade-off between energy efficiency and service level in real environments. The classification of VMs required by our algorithm could be easily inferred by specific information embedded by service developers into software templates, meta-models, and annotations, which are becoming conventional development paradigms for cloud applications [7].

The paper is organized as follows. Initially, Section II reviews related work. Section III describes our system model, while Section IV defines the optimization problem for workload consolidation. Then, Section V discusses the consolidation procedure. Section VI shows numerical results. Finally, Section VII gives our conclusions and plans for future work.

II. Related Work

Resource consolidation is one of the most effective ways to save energy in data centers. It implies gathering VMs

[1]Sleeping (i.e., Advanced Configuration and Power Interface – ACPI S3 state) consists of very low energy consumption states for electronic devices, where most functions are disabled but the system is able to resume to full operation in a very short time (typically under a few hundred milliseconds).

978-1-5386-7177-1/18/$31.00 ©2018 IEEE

on the minimum number of servers needed to fulfill their performance requirements (CPU, RAM, Input/Output – I/O bandwidth), and to shut down the remaining infrastructure.

The consolidation problem has often been modeled as a modified *bin packing* problem, seeking the optimal operating point between energy and performance [8]. However, VM placement is not exactly equivalent to a three-dimensional (CPU, RAM, I/O) bin packing problem, and some anomalies could arise from this approach and similar ones in the literature [9]. Given the NP-completeness of the problem, most authors end up proposing heuristics to find a sub-optimal solution in acceptable time [8]–[10].

Several researchers have tackled the trade-off between traffic routing and power consumption in data center networks. Heller *et al.* [11] model the consolidation of network flows as a Multi-Commodity Flow Problem (MCFP) and a greedy bin packing. In other cases, VMs are placed so to minimize the network power consumption. Shirayanagi *et al.* [12] target fault-tolerance considering the placement of back-up VMs.Wang *et al.* [13] cluster VMs according to their traffic patterns, and then place them on different racks with a classical k-cut algorithm; finally, the minimal number of network switches to carry the total traffic is computed, assuming a balancing algorithm is used (VLB/ECHP).

The problem of consolidation has also been modeled by jointly considering the power consumption of servers and network equipment. Kliazovich *et al.* [14] define a metric based on a hierarchical model that accounts for energy dissipation of servers and network equipment, which is used to find the best balance between server utilization and queue size. Instead, Fang *et al.* [15] decompose the problem into three parts:

(1) *traffic*-aware grouping modeled as a Balanced Minimum k-cut Problem;
(2) *distance*-aware VM-group to server-rack mapping modeled as a Quadratic Assignment Problem;
(3) *power*-aware inter-VM traffic flow routing defined as a classic MCFP.

Specific heuristics have been used to find a solution for each part.

We remark that most of the proposed approaches only take into consideration static resource constraints. Just few works [10, 16] have designed consolidation strategies that estimate over/under loading conditions on each server. However, these consolidation strategies are power-unaware, since they only consider the utilization metric. Overall, this approach achieves better efficiency, but takes decision based on a delayed context, because the estimation of resource utilization takes some time, hence leading to potential violations of the Service Level Agreement (SLA). We argue that service users should be aware of any potential performance violations, and should be rewarded for their willingness to participate in more aggressive power-saving mechanisms (for instance, by dynamic and context-aware pricing schemes [17]).

III. System Model

The system model of a cloud environment represents its elements, their dependencies and relationships. We can split the description according to the logical separation between physical infrastructure and virtual resources (i.e., VMs). In addition, for the purpose of optimization, we also need a model for power consumption of the whole system. The proposed request model differs from other works [18] mainly in consideration of the *time* dimension.

A. Data Center Infrastructure Model

We consider a data center made of physical servers, network switches, and links. We don't impose any specific architecture or topology for the network. We model a data center as a weighted directed graph $\mathcal{G}(\mathbb{M}, \mathbb{E})$, where $\mathbb{M}$ and $\mathbb{E}$ are sets of nodes and edges, respectively. The set of nodes includes both servers (leafs in $\mathcal{G}$), which build the subset $\mathbb{S}$, and network switches (roots and internal nodes in $\mathcal{G}$), which build the subset $\mathbb{N}$. Obviously, $\mathbb{M} \equiv \mathbb{S} \cup \mathbb{N}$. The directed edge $(n, m) \in \mathbb{E}$ represents a network link from node n to m where $n, m \in \mathbb{M}$.[2]

Resources available to a server $n \in \mathbb{S}$ are defined by a pair composed of the number of CPUs[3] c_n and of the quantity of RAM r_n, expressed in MB. Network resources are given in terms of link capacity b_{nm} (expressed in Mbps with $n, m \in \mathbb{M}$) and number of available ports σ_n for each network device $n \in \mathbb{N}$.

For each node $n \in \mathbb{M}$, we define the utilization level $\dot{u}_n$. This metric is proportional to the fraction of CPUs allocated to VMs for servers, and to the number of used ports for network devices. In addition, we define a metric for link utilization as the current bandwidth reserved $\vec{u}_{nm} \in [0, 1]$ by all network paths that include that link.

Finally, we define a network (routing) path p_{nm} as a finite sequence of distinct directed edges from the source server $n \in \mathbb{S}$ to the destination one $m \in \mathbb{S}$. We also denote with $\mathbb{P}_{nm}$ the set of all those paths, while $\mathbb{P} = \bigcup_{n,m \in \mathbb{S}} \mathbb{P}_{nm}$ refers to the set of all the networks paths. The cardinality of these sets depends on the network topology: based on the redundancy level, there will be multiple paths between two servers.

All the notation introduced in this section are summarized in Table I.

B. Virtualization Model

Every application consists of one or more elementary software components, each deployed in a separate VM, which represents the granularity of our model. We do not need to explicitly take into account the belonging of VMs to specific applications since this aspect is already captured by the communication patterns.

The model defines two different sets of VMs:

(i) $\mathbb{V}(t)$ includes all VMs running in the data center at the instance t;

[2]Considering this node classification, we also refer to $n \in \mathbb{M}$ and to $(n, m) \in \mathbb{E}$ (where $n, m \in \mathbb{M}$) with the terms *device* and *link*, respectively.

[3]For the sake of simplicity, in this paper, the term CPU refers to the minimum processing unit that can be set in a low-consumption state.

TABLE I: Data Center Infrastructure Model Notation.

Parameter	Description
$\mathcal{G}(\mathbb{M},\mathbb{E})$	Weighted directed graph model for the data center infrastructure.
$\mathbb{M}$	Set of nodes (servers and network devices).
$\mathbb{S}$ ($\mathbb{N}$)	Sets of servers (network devices).
$(n,m) \in \mathbb{E}$	Directed edge from node $n \in \mathbb{M}$ to $m \in \mathbb{M}$.
c_n (r_n)	Number of CPUs (amount of RAM in MB) available in server $n \in \mathbb{S}$.
σ_n	Number of network ports of switch $n \in \mathbb{N}$.
b_{nm}	Link capacity (in Mbps) of the edge from nodes $n \in \mathbb{M}$ to $m \in \mathbb{M}$.
$\vec{u}_{nm} \in [0,1]$	Bandwidth utilization of the edge from node $n \in \mathbb{M}$ to $m \in \mathbb{M}$.
$\dot{u}_n$	Utilization level of node $n \in \mathbb{M}$.
$p_{nm} \in \mathbb{P}_{nm} \subseteq \mathbb{P}$	Network path from server $n \in \mathbb{S}$ to $m \in \mathbb{S}$ with no repeated edges.
$\mathbb{P}_{nm}$	Set of network paths from server $n \in \mathbb{S}$ to $m \in \mathbb{S}$ with no repeated edges.
$\mathbb{P} = \bigcup_{n,m \in \mathbb{S}} \mathbb{P}_{nm}$	Set of all possible network paths.

TABLE II: Virtualization Model Notation.

Parameter	Description
$\tilde{c}_v$ $(\tilde{r}_v) \in \mathcal{R}_{\geq 0}$	Number of CPUs (amount of RAM in MB) required by the VM $v \in \mathbb{V}$.
$\tilde{b}_{vw} \in \mathcal{R}_{\geq 0}$	Bandwidth (in Mbps) required between VMs $v \in \mathbb{V}$ to $w \in \mathbb{V}$.
t_v^{a} $(t_v^{\mathrm{d}}) \in [0,T]$	Arrival (duration) time (in second) of the VM $v \in \mathbb{V}$.
$x_v \in \{\mathbb{S} \cup \emptyset\}$	Server where the VM $v \in \mathbb{V}$ is placed or empty value in case of a request rejection.
$\mathbf{x} \coloneqq [x_v \mid v \in \mathbb{V}]$	Vector of VM server assignments.
$y_{vw} \in \{\mathbb{P}_{nm} \cup \emptyset\}$	Network path for the communication from VMs $v \in \mathbb{V}$ to $w \in \mathbb{V}$ (with $x_v = n$ and $x_w = m$) or empty value in case of a request rejection.
$\mathbf{y} \coloneqq [y_{vw} \mid v,w \in \mathbb{V}]$	Vector of network path assignments.

(ii) $\mathbb{V}_k$ includes the VMs from the *incoming* request $k \in \mathbb{R}$. For the sake of simplicity, we use the plain notation $\mathbb{V}$ when a definition may be applied to both sets (e.g., the notation in Table II).

VMs are created and destroyed according to the application life-cycle management logic, which detail is not relevant, and is modeled as arrival/departure processes that follow an exponential distribution [18, 19]. The request specified by a tenant arrives at a generic instant $t \in [0,T]$ and consists of a new set $\mathbb{V}_k$ of VMs to allocate.

A consolidation plan defines the placement of VMs on servers, and the reservation of bandwidth on network paths. The parameter $x_v \in \{\mathbb{S} \cup \emptyset\}$ indicates in which server the VM $v \in \mathbb{V}$ is placed; an *empty* value ($\emptyset$) means that either the placement procedure must still be completed or it has failed (rejected request). The vector $\mathbf{x} \coloneqq [x_v \mid v \in \mathbb{V}]$ includes all the placement assignments.
The parameter $y_{vw} \in \{\mathbb{P}_{nm} \cup \emptyset\}$ indicates the selected network path for communication from $v \in \mathbb{V}$ to $w \in \mathbb{V}$ (with $x_v = n$ and $x_w = m$); an empty value ($\emptyset$) is used if the network path selection must still be completed or it has failed (rejected request). The vector $\mathbf{y} \coloneqq [y_{vw} \mid v,w \in \mathbb{V}]$ includes all the network path assignments.

Our model for VMs includes QoS/SLA requirements (i.e., requested computing/networking resources: CPU, RAM, end-to-end bandwidth) and context information (sensitivity to violations of QoS/SLA and current usage pattern). We define three distinct QoS classes, labeled as:

(a) *red* for very critical entities (e.g., they are currently processing medium/high loads, and/or there are tight QoS constraints on availability, processing, bandwidth, latency);
(b) *yellow* for busy entities that are not operating close to their performance limit (e.g., they are currently processing low loads, and QoS constraints are largely met);
(c) *green* for low-load entities (e.g., they are not currently processing anything, and there are negligible risks of SLA violations).

We formalize the set of classes as $\mathbb{C} \coloneqq \{\alpha_{\mathrm{r}}, \alpha_{\mathrm{y}}, \alpha_{\mathrm{g}}\}$. The class to which a specific VM $v \in \mathbb{V}$ belongs is indicated by $\delta_v \in \mathbb{C}$. For each class $h \in \mathbb{C}$, we indicate with f_h the fraction of CPU guaranteed to the VMs. We assume shared and unlimited network storage: the resources by the VM do not include storage requirements.

As noted in a previous section, our model can be seen as extension of the work by Dalvandi *et al.* [18]. However, unlike that work, the time dimension is a continuous domain where the request can include VMs with different duration times.

C. Power Model

We use two power states for servers and switches:

(1) *standby* or *sleep* ($p_\star$), equivalent to the ACPI S3 state (also known as *Suspend-to-RAM*);
(2) *active* ($p_\bullet$), when all the hardware is switched-on.

The current power state $\Phi_n^\star$ depends on the device utilization:

$$p_n = \begin{cases} p_\star & \text{if} \quad \dot{u}_n = 0 \\ p_\bullet & \text{if} \quad \dot{u}_n > 0 \end{cases} \tag{1}$$

Though the power state is not directly involved in the consolidation model, it represents the main outcome from the optimization process.

We consider a utilization-based power model, which captures the typical non-linear behavior for ICT devices [20, 21]:

$$\Phi(\dot{u}_n) = \begin{cases} \Phi_n^\star & \text{if} \quad \dot{u}_n = 0 \\ \Phi_n^\circ + \dot{u}_n \cdot (\Phi_n^\bullet - \Phi_n^\circ) & \text{if} \quad \dot{u}_n > 0 \end{cases} \tag{2}$$

where $\Phi_n^\star$, Φ_n° are the standby and idle power consumption, respectively; $\Phi_n^\bullet$ is the power consumption at full load.

Table III summarizes all the notation introduced in this section.

IV. Consolidation Model

A. Objective

The ultimate goal of consolidation is to minimize the total power consumption of the data center, by proper identifica-

TABLE III: Power Model Notation.

Parameter	Description
p_n	Power state of the device $n \in \mathbb{M}$.
$p_\star$ ($p_\bullet$)	*Standby* (*Active*) power state
$\Phi(\dot{u}_n)$	Power consumption of the device $n \in \mathbb{M}$.
$\Phi_n^\star$ (Φ_n°)	Standby (Idle) power consumption of the device $n \in \mathbb{M}$.
$\Phi_n^\bullet$	Power consumption of the device $n \in \mathbb{M}$ at full load.

tion of the best *placement of VMs* on servers and *network path selection*:

$$\min_{\mathbf{x},\mathbf{y}} \sum_{n \in \mathbb{M}} \Phi(\dot{u}_n) \tag{3}$$

under the constraints on available resources (Sub-section III-B). Based on the utilization computed for each node, a power state is assigned accordingly by (2).

The placement problem consists in identifying the group of servers where VMs can be allocated, with the objective to minimize the total power consumption of servers. The selection of the network paths has the objective to minimize the number of active switches. The two processes are correlated (network paths originate and terminate on the servers where VMs are placed), and this generates additional constraints that we have to formalized in the consolidation model.

The main novelty of our work consists in defining QoS classes for VMs. Roughly speaking, our target is the design of a consolidation strategy that:

(i) keeps *red* VMs on the smallest number of servers, while assuring enough resources are provisioned to meet their stringent QoS requirements;
(ii) groups *yellow* VMs together on the smallest number of servers, even on the same servers as *red* VMs if enough resources are available;
(iii) distributes *green* components among all servers, with no stringent constraints on grouping the largest number of them together, since servers hosting *green* components only will sleep;
(iv) limits the number of migrations, so to avoid as much as possible service disruption and performance degradation.

Again, these objectives are formalized as additional constraints.

B. Constraints

Before listing the full list of constraints for the optimization of the cost function, we need to introduce some more notation (see Table IV). The current placement of VMs on servers and set of network paths are stored in the variables $\dot{x}_v$ (with $v \in \mathbb{V}(t)$) and $\dot{y}_{vw}$ (with $v, w \in \mathbb{V}(t)$), respectively. Similarly to what described in Section III-B, these two variables are included in the vectors $\dot{\mathbf{x}}$ and $\dot{\mathbf{y}}$.
In addition, we explicitly identify the need for migrations of VMs (i.e., $x_v \neq \dot{x}_v$) and re-routing of network traffic (i.e., $y_{vw} \neq \dot{y}_{vw}$) by two binary variables $\vec{x}_v$ and $\vec{y}_{vw}$, respectively. Note that network paths automatically change alongside the migration of the corresponding VMs, but may also change

TABLE IV: Consolidation Model Notation.

$\dot{x}_v \in \mathbb{S}$	Current server hosting VM $v \in \mathbb{V}(t)$.
$\dot{\mathbf{x}} \coloneqq [\dot{x}_v \mid v \in \mathbb{V}(t)]$	Vector of servers hosting VMs.
$\dot{y}_{vw} \in \mathbb{P}_{nm}$	Current network path between VMs $v \in \mathbb{V}(t)$ and $w \in \mathbb{V}(t)$ (with $\dot{x}_v = n$ and $\dot{x}_w = m$).
$\dot{\mathbf{y}} \coloneqq [\dot{y}_{vw} \mid v, w \in \mathbb{V}(t)]$	Vector of network path assignments.
$\vec{x}_v \in \{0, 1\}$	Indicates if the VM $v \in \mathbb{V}$ has to be migrated from a server to another one.
$\vec{y}_{vw} \in \{0, 1\}$	Indicates if the selected network path for the communication from VMs $v \in \mathbb{V}$ to $w \in \mathbb{V}$ is changed.
$\vec{x}_*$ ($\vec{y}_*$)	Upper bound on the number of migrations (network paths changes).

following the arrival, departure, or migration of other VMs, according to bandwidth needs. Since minimal service disruption is one explicit target of our consolidation framework, we fix an upper bound to the number of migrations ($\vec{x}_*$) and re-routing of network paths ($\vec{y}_*$).

The full set of constraints is given by (4–15). To simplify the notation, (4) defines the reference set of VMs $\mathbb{V}_*$.

$$\mathbb{V}_* = \mathbb{V}(t) \cup \mathbb{V}_k \qquad k \in \mathbb{R} \tag{4}$$

$$x_v \in (\mathbb{S} \cup \emptyset) \qquad \forall v \in \mathbb{V}_* \tag{5}$$

$$y_{vw} \in (\mathbb{P}_{nm} \cup \emptyset) \qquad \forall v, w \in \mathbb{V}_* \wedge\ n \neq m \wedge x_v = n \wedge\ x_w = m \tag{6}$$

(5–6) represent the constraints related to the domains of the problem variables x_v and y_{vw}. When the number of available resources is not enough to satisfy the VM requirements, it is impossible to place every VM on a server. Hence, the domains of the problem variables include the empty value ($\emptyset$), too.

$$\sum_{v \in \mathbb{V}_* \wedge\ x_v = n\ \wedge\ \delta_v = h} \tilde{c}_v \times f_h \leq c_n \qquad \forall n \in \mathbb{S} \tag{7}$$

$$\sum_{v \in \mathbb{V}_* \wedge\ x_v = n} \tilde{r}_v \leq r_n \qquad \forall n \in \mathbb{S} \tag{8}$$

$$\sum_{v,w \in \mathbb{V}_* \wedge\ \exists (n,m) \in y_{vw}} \tilde{b}_{vw} \leq b_{nm} \qquad \forall (n, m) \in \mathbb{E} \tag{9}$$

The next two equations (7–8) ensure that the VMs are placed on servers with sufficient available space for the requested resources (CPU and RAM, respectively). Over-commitment on CPU is possible, according to the specific QoS class the VM belongs to; for this reason, the factor f_h (with $\delta_v = h$) scales CPU requirements for each VM $v \in \mathbb{V}_*$.

$$\vec{x}_v = \begin{cases} 1 & \text{if} \quad x_v \neq \dot{x}_v \\ 0 & \text{else} \end{cases} \qquad \forall v \in \mathbb{V}(t) \tag{10}$$

$$\vec{y}_{v,w} = \begin{cases} 1 & \text{if} \quad y_{vw} \neq \dot{y}_{vw} \\ 0 & \text{else} \end{cases} \qquad \forall v, w \in \mathbb{V}(t) \tag{11}$$

(9) guarantees that the total bandwidth requested on each link is less or equal to the link capacity. The limits on the number

of migrations and changes to network paths are imposed by (10–12), respectively.

$$\sum_{v \in \mathbb{V}_*} \vec{x}_v \leq \vec{x}_* \qquad \sum_{v,w \in \mathbb{V}_*} \vec{y}_{vw} \leq \vec{y}_* \tag{12}$$

Finally, (13–15) are used to compute the utilization level of the servers, network devices and edges, respectively. For the servers, the utilization level is a fraction of the CPU utilized by the VMs, for the network devices it is a fraction of the available ports, and for the edges it is the fraction of the reserved bandwidth by VMs belonging to the *red* and *yellow* classes. We do not consider the VMs belonging to the green class because they are idle and do not exchange data.

$$\dot{u}_n = \frac{\sum_{v \in \mathbb{V}_* \wedge\ x_v = n \wedge\ \delta_v = h} \tilde{c}_v \times\ f_h}{c_n} \qquad \forall n \in \mathbb{S} \tag{13}$$

$$\dot{u}_n = \frac{1}{\sigma_n} \times \sum_{(n,m) \in \mathbb{E}} \operatorname{sgn}\ (\vec{u}_{nm} + \vec{u}_{mn}) \qquad \forall n \in \mathbb{N} \tag{14}$$

$$\vec{u}_{nm} = \frac{b_{nm} - \sum_{\substack{v,w \in \mathbb{V}_* \wedge \\ \exists (n,m) \in y_{vw} \\ \wedge\ \delta_v, \delta_w \neq\ \alpha_{\mathrm{g}}}} \tilde{b}_{vw}}{b_{nm}} \qquad \forall (n,m) \in \mathbb{E} \tag{15}$$

C. Search methods and Heuristics

Despite the complexity and the dimension of the problem, we followed a different approach than other related works in the literature [3, 9, 15]. We formulate the proposed consolidation strategy as a global optimization procedure, which solves the placement and routing problem simultaneously.

Clearly, we can neither find a closed-form solution nor performing an exhaustive search (since the problem is NP-hard). For this reason, we looked for suitable and efficient heuristic methods, trying to obtain a sub-optimal solution as close as possible to the optimal one in a reasonable period of time. Specifically, we used the Optaplanner constraint satisfaction solver [22]. This framework supports 3 families of optimization algorithms:

(1) Exhaustive Search (ES);
(2) Construction Heuristics (CH);
(3) Meta-Heuristics (MH).

The algorithms of the ES family always find the global optimum and recognize it too. However, they do not scale (not even beyond small data sets) and they are mostly useless. As a matter of fact, they suffer scalability issues from the point of view of memory and performance. We consider the Meta-Heuristics family (in combination with Construction Heuristics ones for the initialization) which are the recommended choice to deliver a good result in a reasonable period of time. In more details, the CH builds a pretty good initial solution in a finite length of time. Its solution is not always feasible, but it finds it fast so MH can finish the job. The heuristic types considered in this work are:

(i) *Hill Climbing* (HC) [23];
(ii) *Tabu Search* (TS) [24, 25];
(iii) *Simulated Annealing* (SA) [26];
(iv) *Late Acceptance* (LA) [27].

V. Consolidation Algorithm

The pseudo-code for the consolidation procedure is shown in Algorithm 1. It computes new placement and routing plans according to resource availability; in case the requests exceed the capacity, a simple admission control selectively discards some VMs. The algorithm is executed periodically or after the reception of a new request $k \in \mathbb{R}$ at instance t.

The strategy starts by removing the terminated VMs from the set $\mathbb{V}(t)$ (line 2). The *optimize* function (line 4) computes a new allocation plan which contains the placement of VMs on servers and the network paths for the communication between VMs. If a solution is not found, the consolidation is tried again after removing some VMs from the request set $\mathbb{V}_k$ (line 5). The procedure is repeated until either a solution is found or all the incoming VMs are discarded (lines 3-8). In this case, the algorithm returns the original plan and no changes occur.

RemoveTerminatedVM removes the *expired* VMs from the set $\mathbb{V}(t)$: i.e. the ones with a completion time (computed as $t_v^{\mathrm{a}} + t_v^{\mathrm{d}}\ \forall v \in \mathbb{V}(t)$) lower than current time t.

The Optimize function computes new placement and routing plans, by minimizing the total power consumption of servers and network devices (see Section IV). This function requires different input parameters. It needs the sets of VMs $\mathbb{V}(t)$ and $\mathbb{V}_k$ for a request $k \in \mathbb{R}$. Obviously, for periodic executions that do not correspond to the arrival of a new request, $\mathbb{V}_k$ is empty. The vectors $\mathbf{x}$ and $\mathbf{y}$ store the new allocation plans computed by the optimization. Instead, the vectors $\dot{\mathbf{x}}$ and $\dot{\mathbf{y}}$ store current placement of VMs on servers and network paths, respectively. Finally, in order to compute the new plan, this function requires the sets describing the infrastructure (devices $\mathbb{M}$ and links $\mathbb{E}$), the classes $\mathbb{C}$, and all possible network paths $\mathbb{P}$. Since the total amount of resources requested by the users may exceed the system capacity, we need an admission control to avoid that excessive over-commitment lead to violation of QoS and SLA. CutRequest discards VMs according to their class and demand in terms of resources. To this aim, we assign an increasing priority to the classes in the set $\mathbb{C}$ according to the following order:

(1) *green*;
(2) *yellow*;
(3) *red.*

We first select the VMs from $\mathbb{V}_k$ (with $k \in \mathbb{R}$) that belong to the class with the lowest priority. Among them, we select

Algorithm 1 Consolidation

```
1: function Consolidation(V(t), V_k, x, y, ẋ, ẏ, M, E, C, P)
2:   V(t) ← RemoveTerminatedVM(V(t))
3:   do
4:     if Optimize(V(t), V_k, x, y, ẋ, ẏ, M, E, C, P) = ∅ then
5:       V_k ← CutRequest(V_k, C)
6:     else
7:       break
8:   while V_k ≠ ∅
9:   return x, y, ẋ, ẏ, M, E, P
```

VMs with the highest number of required CPUs. If more than one candidate are still present, we choose the ones with the greatest required amount of RAM. Finally, if the selected VMs are still more than one, we randomly choose one of them to discard.

VI. Performance Evaluation

In this section, we validate the effectiveness of the proposed algorithm through simulations. We investigate how different heuristics affect the efficiency of the consolidation algorithm, by considering complementary metrics as energy-saving, and acceptance rate.

A. Simulation Settings

We consider a typical multi-rooted hierarchical topology for the Data-Center Network (DCN), with three layers of switches (Core, Aggregation, and Edge). A group of servers in a rack is connected to an Edge switch, which in turn is connected to Aggregation switches. Each Aggregation switch is connected to a group of Core switches, which are attached to some front-end servers for connecting to external clients. In particular, we choose a fat-three topology [28]. The total number of switches and servers are equal to 80 and 128, respectively, and the link capacity is set to 1 Gbps. We assign the same amount of resources to each server: 128 GB of RAM and 32 CPUs.

Table V shows the values of power consumption for servers and network devices. We chose the values according to references [11, 21]. Since the power consumption for servers has a direct relationship with the number of cores [21], we scale up these values based on the number of resources. In our simulations, we considered servers with 10 CPUs.

We simulate the data center operation for $T = 24$ hours, so to capture a typical daily workload. Unlike other works [18], the simulation time domain is continuous. Hence, a new request can arrive at any time between 0 and 24 hours. The mean arrival rate of the incoming requests follows the typical daily profile as shown in Fig. 1 [29]. The minimum value of 400 requests per second occurs overnight around 00:00. During the day, the mean request rate gradually grows to reach the maximum value of 1200 requests per seconds around 12:00. From the afternoon, the average rate decreases until reaching the minimum value again.

We use the truncated Log-normal distribution (tLN) for generating the number of VMs per request [18]. We vary the mean request duration in the range $[1, 10]$ seconds using a tLN with mean value equal to 5 seconds. The number of CPUs required by a VM follows a tLN in the range $[0, 10]$ with mean equal to $\bar{c} = 3$. The RAM required by a VM follows a tLN in the range $[1, 10]$ GB with mean $\bar{r} = 20$ GB. Finally, the bandwidth required by a VM is also generated by a tLN in the range $[0, 200]$ Mbps, with mean $\bar{b} = 100$ Mbps.

The VM class is assigned based on the following probability: *red* – 20%, *yellow* – 50%, and *green* – 30%. Instead, for what concerns the classes $\mathbb{C}$ and their relative properties f_h we consider the following values: 1.00 CPU, 0.20 CPU, and 0.00 CPU for *red*, *yellow*, and *green*, respectively.

TABLE V: Power Consumption Values.

Device $n \in \mathbb{M}$	Φ_n°	$\Phi_n^{\bullet}$	$\Phi_n^{\star}$
Server	2720 W	3520 W	≈ 5 W
Edge Switch	76.40 W	102 W	
Aggregation Switch	133.50 W	175 W	
Core Switch	555.00 W	656 W	

Finally, regarding the parameters for the optimization, the upper bounds for the migrations of VMs and the re-routing of traffic between VMs are set to 25% and 15%, respectively.

B. Metrics

Performance evaluation investigates the effectiveness of the consolidation algorithm for all the heuristics described in Sub-section IV-C. We take into account the following metrics:

(i) *energy savings* with respect to power-unaware consolidation;
(ii) *acceptance rate*, i.e. average percentage of the accepted VM requests per second.

Energy saving is the explicit optimization target for the consolidation procedure. The set of constraints guarantees the correct level of over-commitment on processing and networking resources. However, heuristics do not fully explore the space of feasible solutions, so one method may provide better solutions than others. For comparison, we provide energy saving with respect to a more traditional consolidation approach, which does not consider power consumption. It first clusters VMs on the smallest number of servers, and than computes routing paths; the same constraints on computing resources and network bandwidth are taken into account.

The behavior or the heuristic also impacts performance. In particular, depending on the placement and routing plans, there may be a sub-optimal allocation of resources, which does not allow to satisfy all the requests. For this reason, we explicitly consider the acceptance rate, as a sort of *penalty* introduced by heuristic searches.

C. Numerical Results

We wrote an event simulator in Java using the Optaplanner constraint satisfaction solver to evaluate the proposed algorithm. The validation takes into account different heuristics to solve the optimization problem (see Section IV-C), and

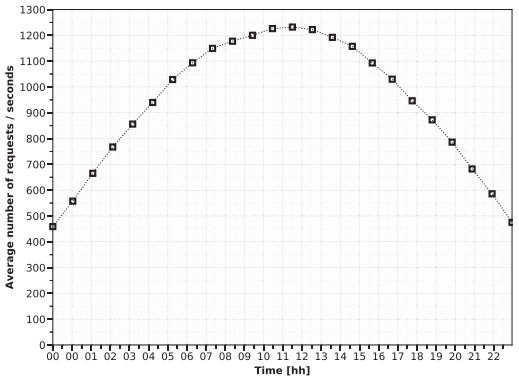

Fig. 1: Daily profile of the average number of requests per second.

their effectiveness to save energy and maximize the usage of the Data center (i.e., to accept the largest number of VMs).

We repeated the experiments for a sufficient number of times to get small errors, representing the 95% Confidence Intervals (CIs). The measured CIs have not been reproduced in the graphs since all of them are small and clutter the figures.

Fig. 2 displays the average percentage of the energy saving throughout the day, with respect to power-unaware consolidation (see Section VI-B). The trend shows that great saving is achieved when the workload (i.e., the number of VMs) is low. This is expected, because with higher load there are less idle resources and less opportunity to apply power-saving mechanisms. It is the *linear* relationship between utilization and power consumption at the base of efficient computing.

The results show a maximum energy saving of more than 50% with the LA heuristic. Compared to the other methods, HC obtains worse results because it easily gets stuck in local optima. Instead, the other heuristics implement more advanced solutions to avoid this problem. With a low number of requests, in the LA case, the energy saving is stably more than 30%. The trend during the simulation has far larger variance for the TS, SA and LA heuristics than the HC. The former methods exploit the concept of *perturbations*, which results in better exploration of the domain. The acceptance rate in shown in Fig. 3. The LA method performs better than the other heuristics. Unlike the metric of energy saving, the variance is large for HC, too. Obviously, the problem of the local minima also impacts the acceptance rate. For high values of incoming requests, HC presents an acceptance rate below 74% compared to 86% of the TS case. SA behaves relatively better than LA (especially with a high number of requests). Most likely, the advanced solutions introduced by LA [27] allow obtaining better results from the point of view of optimization (less power consumption) at the cost of a worse allocation of the resources.

The impact of the energy-efficiency constraints in the acceptance ratio is shown in Fig. 4. The reduction of the acceptance ratio compared to the power-unaware consolidation is mainly less than 1%, except for the isolated case with the LA heuristic. As described above, this heuristic shows

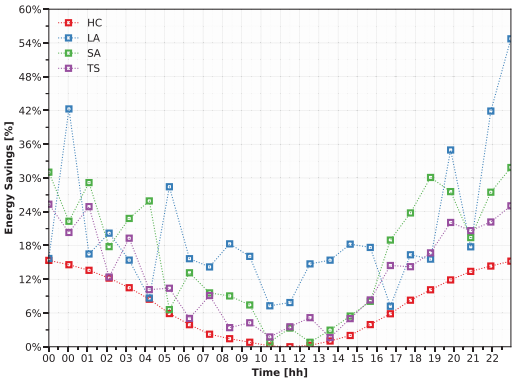

Fig. 2: Average percentage of energy saving.

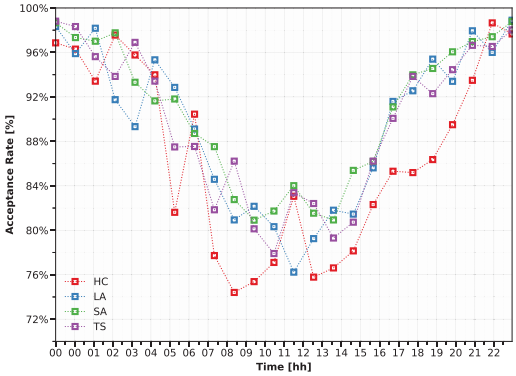

Fig. 3: Average percentage of accepted requests.

a better energy saving compared to the other ones at the expense of reducing the acceptance ratio.

VII. Conclusions

In this paper, we have investigated workload consolidation for energy efficient data center operation. We have introduced a novel dimension in the optimization problem, by considering the presence of multiple classes of VMs, related to the current context (e.g., to different roles in the overall service/application). The class does not only affect over-commitment, but also provides indication about usage of VMs.

Our approach enables more aggressive power saving to be applied in data centers, by allowing to sleep servers that only host *spare* software components. We have shown that our consolidation strategy improves the efficiency of the infrastructure, with minor implications on the acceptance rate of additional service requests.

Future works will include:

(1) inclusion of a tighter integration between the server and network models, to achieve better savings;
(2) take into account the power consumption of network devices for different operating rates;
(3) detailed study and analysis of further heuristics, to find optimal allocations even with larger infrastructures.

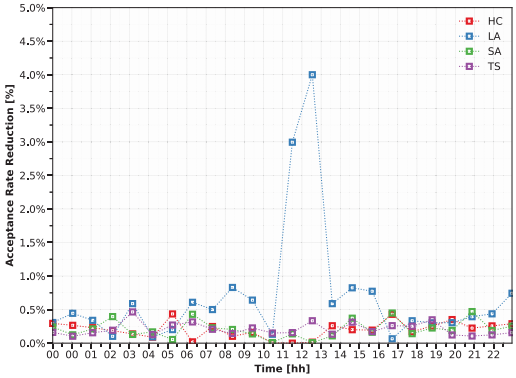

Fig. 4: Reduction of accepted requests with respect to power-unaware consolidation.

ACKNOWLEDGMENT

This work was partially supported by the European Commission under the projects ARCADIA (grant no. 607881) and MATILDA (grant no. 761898).

REFERENCES

[1] L. A. Barroso and U. Hlzle, "The Case for Energy-Proportional Computing," *Computer*, vol. 40, no. 12, pp. 33–37, Dec. 2007.

[2] A. Hammadi and L. Mhamdi, "A Survey on Architectures and Energy Efficiency in Data Center Networks," *Comp. Comm.*, vol. 40, pp. 1–21, Mar. 2014.

[3] A. Stage and T. Setzer, "Network-Aware Migration Control and Scheduling of Differentiated Virtual Machine Workloads," in *Proc. of the Wksh. on Software Engin. Chall. of Cloud Comput. (ICSE)*, Vancouver, Canada, May 23, 2009, pp. 9–14.

[4] G. A. Brady, N. Kapur, J. L. Summers, and H. M. Thompson, "A case study and critical assessment in calculating power usage effectiveness for a data centre," *Energy Conversion and Management*, vol. 76, p. 155161, December 2013.

[5] J. Yuventi and R. Mehdizadeh, "A critical analysis of power usage effectiveness and its use in communicating data center energy consumption," *Energy and Buildings*, vol. 64, pp. 90–94, September 2013.

[6] R. Bolla, L. Sambolino, D. Tigano, and M. Repetto, "Enhancing Energy-Efficient Cloud Management through Code Annotations and the Green Abstraction Layer," in *Proc. of the 1st Intl. Wksh. on Sustain. Data Centres and Cloud Comput. (SD3C)*, Limassol, Cyprus, Dec. 7, 2015.

[7] "Topology and Orchestration Specification for Cloud Applications," OASIS Std., Nov. 2013, vers. 1.0. [Online]. Available: http://docs.oasis-open.org/tosca/TOSCA/v1.0/os/TOSCA-v1.0-os.pdf

[8] S. Srikantaiah, A. Kansal, and F. Zhao, "Energy Aware Consolidation for Cloud Computing," in *Proc. of the Wksh. on Pow. Aware Comp. and Syst. (ICAC)*, San Diego, CA, U.S.A., Dec. 7, 2008.

[9] M. Mishra and A. Sahoo, "On Theory of VM Placement: Anomalies in Existing Methodologies and Their Mitigation Using a Novel Vector Based Approach," in *Proc. of the 4th IEEE Intl. Conf. on Cloud Comput. (CLOUD)*, Mumbai, India, Jul. 4–9, 2011, pp. 275–282.

[10] A. Beloglazov and R. Buyya, "OpenStack Neat: a Framework for Dynamic and Energy-Efficient Consolidation of Virtual Machines in OpenStack Clouds," *Concur. and Computa.: Pract. and Exp. (CCPE)*, vol. 27, no. 5, pp. 1310–1333, Apr. 2015.

[11] B. Heller, S. Seetharaman, P. Mahadevan, Y. Yiakoumis, P. Sharma, S. Banerjee, and N. McKeown, "ElasticTree: Saving Energy in Data Center Networks," in *Proc. of the 7th USENIX Conf. on Net. Syst. Des. and Impl. (NSDI)*. Berkeley, CA, U.S.A.: USENIX Assoc., 2010, p. 17.

[12] H. Shirayanagi, H. Yamada, and K. Kono, "Honeyguide: A VMMigration-Aware Network Topology for Saving Energy Consumption in Data Center Networks," in *Proc. of the 17th IEEE Symp. on Comp. and Comm. (ISCC)*. Cappadocia, Turkey: IEEE, Jul. 1–4, 2012, pp. 460–467.

[13] L. Wang, F. Zhang, A. V. Vasilakos, C. Hou, and Z. Liu, "Joint Virtual Machine Assignment and Traffic Engineering for Green Data Center Networks," *ACM SIGCOMM Perf. Eval. Rev.*, vol. 41, no. 3, pp. 107–112, Dec. 2013.

[14] D. Kliazovich and P. Bouvry, "DENS: Data Center Energy-Efficient Network-Aware Scheduling," in *Proc. of the IEEE/ACM Intl. Conf. on Green Comput. and Comm. (GreenCom) & Proc. of the IEEE/ACM Intl. Conf. on Cyb., Physic. and Soc. Comput. (CPSCom)*, Hangzhou, China, Dec. 18–20, 2010, pp. 69–75.

[15] W. Fang, X. Liang, S. Li, L. Chiaraviglio, and N. Xiong, "VMPlanner: Optimizing Virtual Machine Placement and Traffic Flow Routing to Reduce Network Power Costs in Cloud Data Centers," *Comp. Net.*, vol. 57, no. 1, pp. 179–196, Jan. 2013.

[16] V. Cima, B. Grazioli, S. Murphy, and T. Bohnert, "Adding Energy Efficiency to Openstack," in *Proc. of the Sustain. internet and ICTfor Sustainab. (SustainIT)*, Madrid, Spain, Apr. 14–15, 2015, pp. 1–8.

[17] C. Wang, N. Nasiriani, G. Kesidis, B. Urgaonkar, Q. Wang, L. Y. Chen, A. Gupta, and R. Birke, "Recouping energy costs from cloud tenants: Tenant demand response aware pricing design," in *Proceedings of the 2015 ACM Sixth International Conference on Future Energy Systems (e-Energy '15)*, Bangalore, India, Jul., 14th-17th, 2015, pp. 141–150.

[18] A. Dalvandi, M. Gurusamy, and K. C. Chua, "Time-Aware VMFlow Placement, Routing, and Migration for Power Efficiency in Data Centers," *IEEE Trans. Netw. Service Manag.*, vol. 12, no. 3, pp. 349–362, Sep. 2015.

[19] T. Benson, A. Anand, A. Akella, and M. Zhang, "Understanding Data Center Traffic Characteristics," *SIGCOMM Comput. Commun. Rev.*, vol. 40, no. 1, pp. 92–99, Jan. 2010.

[20] P. Padala, K. G. Shin, X. Zhu, M. Uysal, Z. Wang, S. Singhal, A. Merchant, and K. Salem, "Adaptive Control of Virtualized Resources in Utility Computing Environments," in *Proc. of the 2nd ACM SIGOPS Europ. Conf. on Comp. Syst. (EuroSys)*. New York, NY, U.S.A.: ACM, 2007, pp. 289–302.

[21] P. Mahadevan, P. Sharma, S. Banerjee, and P. Ranganathan, "Energy Aware Network Operations," in *Proc. of the 28th Intl. Conf. on Comp. Comm. (INFOCOM) Wksh.* Piscataway, NJ, U.S.A.: IEEE, 2009, pp. 25–30.

[22] OptaPlanner - Constraint satisfaction solver (Java™, Open Source). [Online]. Available: http://www.optaplanner.org

[23] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 2nd ed. Pearson Education, 2003.

[24] F. Glover, "Tabu Search – Part I," *ORSA Journ. on Comp.*, vol. 1, no. 3, pp. 190–206, 1989.

[25] ——, "Tabu Search – Part II," *ORSA Journ. on Comp.*, vol. 2, no. 1, pp. 4–32, 1990.

[26] A. Khachaturyan, S. Semenovsovskaya, and B. Vainshtein, "The Thermodynamic Approach to the Structure Analysis of Crystals," *ACTA Cystallogr. Sect. A*, vol. 37, no. 5, pp. 742–754, Sep. 1981.

[27] E. K. Burke and Y. Bykov, "The Late Acceptance Hill-Climbing Heuristic," *Europ. Journ. of Operat. Res.*, vol. 25, no. 1, pp. 70–78, 2017.

[28] M. Al-Fares, A. Loukissas, and A. Vahdat, "A Scalable, Commodity Data Center Network Architecture," *ACM SIGCOMM Comput. Comm. Rev.*, vol. 38, no. 4, pp. 63–74, Aug. 2008.

[29] C. Oppenheimer. (2012) Which is less expensive: Amazon or self-hosted? [Online]. Available: https://gigaom.com/2012/02/11/which-is-less-expensive-amazon-or-self-hosted